SkillBridge | SQL Fundamentals

Read and filter rows

Use SELECT, WHERE and ORDER BY to answer clear questions about a table.

A relational table stores rows with a consistent set of columns. A primary key identifies each row. A foreign key links a row to a related table and helps prevent invalid references.

Imagine courses(id, title, level) with three sample rows. SELECT title FROM courses returns the title column. SELECT title FROM courses WHERE level = 'Beginner' returns only matching rows. Add ORDER BY title to request alphabetical ordering; without ORDER BY, a particular order is not guaranteed.

SELECT COUNT(*) FROM courses counts rows. SELECT level, COUNT(*) FROM courses GROUP BY level counts rows within each level. Choose the columns and grouping that match the question you are answering.

SQL NULL represents missing or unknown information. Use IS NULL or IS NOT NULL to test it rather than comparing with = NULL. A condition involving unknown information can behave differently from a true/false comparison.

Practice: write a query that returns beginner course IDs and titles, sorted by title. Then change it to count beginner courses. Explain why selecting only the needed columns is clearer than using SELECT * everywhere.

Checkpoint: WHERE filters rows; ORDER BY controls the output order. They solve different problems.

Join and change data safely

Connect related tables and use parameters and transactions for data changes.

Suppose enrolments contains user_id and course_id. Join it to courses with SELECT c.title FROM courses c INNER JOIN enrolments e ON e.course_id = c.id WHERE e.user_id = @userId. The join condition connects each enrolment to its course.

An INNER JOIN keeps matching pairs. A LEFT JOIN keeps every row from the left table and uses NULL values where no right-side match exists. Choose deliberately: listing every course, including those with no enrolments, needs a different query from listing only enrolled courses.

INSERT adds a row, UPDATE changes rows and DELETE removes rows. An UPDATE or DELETE without a WHERE condition can affect every row. First inspect the intended target with SELECT using the same condition and practise only in a disposable database.

In C#, write SQL containing a placeholder such as @userId and bind the value with a database parameter. Do not concatenate user input into SQL. Parameters keep values separate from SQL syntax; validation and permission checks are still required.

A transaction groups related changes. Commit when all steps succeed; roll back if a step fails. For a quiz submission, answer rows and the final score should succeed together so a partial result is not saved.

Checkpoint: explain why a valid numeric user ID still requires an ownership check before showing that user's results.

Use a disposable practice folder/database. Original instructional material prepared for SkillBridge.